

Глава 12

Специальные контейнеры

Стандартная библиотека C++ содержит не только контейнеры STL, но и контейнеры, предназначенные для особых целей и предоставляющие простые и понятные интерфейсы. Эти контейнеры можно разделить на так называемые *контейнерные адаптеры*, которые приспособливают стандартные контейнеры STL для особых целей, и битовые множества, представляющие собой контейнеры, содержащие биты или булевы величины.

Существуют три стандартных контейнерных адаптера: стеки, очереди и очереди с приоритетами. В очередях с приоритетами элементы сортируются автоматически в соответствии с критерием сортировки. Таким образом, следующий элемент очереди с приоритетами — это элемент, имеющий наивысшее значение.

Битовое множество — это битовое поле с произвольным, но фиксированным количеством битов. Отметим, что в стандартной библиотеке C++ есть специальный контейнер с переменным размером для хранения булевых значений: `vector<bool>` (см. раздел 7.3.6).

Недавние изменения, связанные со стандартом C++11

Практически все функциональные возможности контейнерных адаптеров были описаны еще в стандарте C++98. Перечислим наиболее важные функциональные возможности, добавленные стандартом C++11.

- Контейнерные адаптеры содержат определения типов `reference` и `const_reference` (см. раздел 12.4.1).
- Контейнерные адаптеры поддерживают семантику перемещения и `rvalue`-ссылки:
 - функция `push()` обеспечивает семантику перемещения (см. разделы 12.1.2 и 12.4.4);
 - исходный контейнер может быть перемещен (см. раздел 12.4.2).
- Контейнерные адаптеры имеют функцию-член `emplace()`, автоматически создающую новый элемент, инициализированный передаваемыми аргументами (см. разделы 12.1.2 и 12.4.4).
- Контейнерные адаптеры имеют функцию-член `swap()` (см. раздел 12.4.4).
- Контейнерные адаптеры позволяют передавать своим конструкторам специальный механизм распределения памяти (см. раздел 12.4.2).

12.1. Стеки

Класс `stack<>` реализует стек (известный как LIFO). С помощью функции-члена `push()` в стек можно вставлять любое количество элементов (рис. 12.1). С помощью

функции-члена `pop()` вставленные ранее элементы можно удалять из стека в обратном порядке (“последним вошел, первым вышел”).

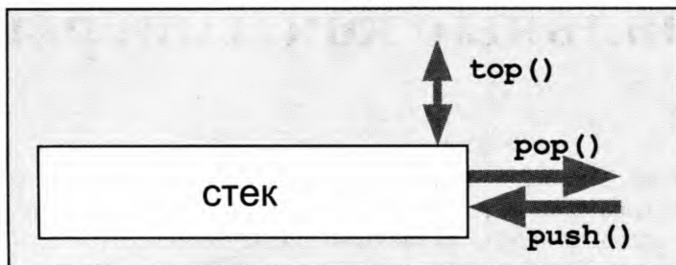


Рис. 12.1. Интерфейс стека

Для использования стека в программу следует включить заголовочный файл `<stack>`.

```
#include <stack>
```

В заголовочном файле `<stack>` класс `stack` определен следующим образом:

```
namespace std {
    template <typename T,
              typename Container = deque<T>>
        class stack;
}
```

Первый шаблонный параметр — это тип элементов. Необязательный второй параметр определяет контейнер, который стек будет использовать для хранения своих элементов. По умолчанию в качестве такого контейнера используется дек. Этот выбор объясняется тем, что в отличие от векторов деки освобождают память при удалении элементов и не копируют все элементы при повторном выделении памяти (см. раздел 7.12, в котором рассматривается вопрос о том, когда и какой контейнер целесообразно использовать).

Например, следующее объявление определяет стек целых чисел:

```
std::stack<int> st; // стек целых чисел
```

Реализация стека просто отображает его операции в соответствующие вызовы контейнера, который внутренне используется стеком (рис. 12.2). Можно использовать любой последовательный контейнерный класс, имеющий функции-члены `back()`, `push_back()` и `pop_back()`. Например, в качестве контейнера можно использовать вектор или список.

```
std::stack<int, std::vector<int>>> st; // стек целых чисел, использующий вектор
```

12.1.1. Основной интерфейс

Основной интерфейс стека состоит из функций-членов `push()`, `top()` и `pop()`.

- Функция `push()` вставляет элемент в стек.
- Функция `top()` возвращает элемент на вершине стека.
- Функция `pop()` удаляет элемент из стека.

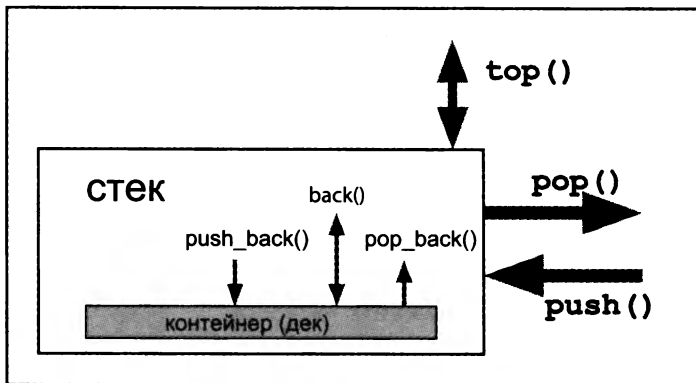


Рис. 12.2. Внутренний интерфейс стека

Отметим, что функция-член `pop()` удаляет следующий элемент, но не возвращает его, а функция-член `top()` возвращает следующий доступный элемент стека, не удаляя его. Таким образом, для обработки и удаления следующего элемента стека следует вызывать обе эти функции. Этот интерфейс несколько неудобен, но он работает лучше, если нужно только удалить следующий элемент, не обрабатывая его¹. Отметим, что поведение функций-членов `top()` и `pop()` для пустого стека не определено. Функции-члены `size()` и `empty()` проверяют, содержит ли стек элементы.

Если вам не нравится стандартный интерфейс класса `stack<>`, можно легко написать более удобный интерфейс. Пример такого интерфейса приведен в разделе 12.1.3.

12.1.2. Пример использования стеков

Следующая программа демонстрирует класс `stack<>`:

```
// contadapt/stack1.cpp

#include <iostream>
#include <stack>
using namespace std;

int main()
{
    stack<int> st;

    // вносим в стек три элемента
    st.push(1);
    st.push(2);
    st.push(3);

    // выводим на печать и удаляем два элемента из стека
    cout << st.top() << ' ';
```

¹ Как указано в книге Г. Саттер *Решение сложных задач на C++* (М.: И.Д. “Вильямс”, 2002), функция, не только удаляющая элемент с вершины стека, но при этом и возвращающая его, оказывается небезопасной в смысле исключений. — *Примеч. консульт.*

```
st.pop();
cout << st.top() << ' ';
st.pop();

// модифицируем элемент на вершине
st.top() = 77;

// вносим два новых элемента
st.push(4);
st.push(5);

// удаляем один элемент, не обрабатывая его
st.pop();

// выводим на печать и удаляем остальные элементы
while (!st.empty()) {
    cout << st.top() << ' ';
    st.pop();
}
cout << endl;
}
```

Эта программа выводит следующий результат:

```
3 2 4 77
```

Отметим, что при работе с нетривиальными типами элементов целесообразно рассмотреть возможность использования функции-члена `std::move()` для вставки элементов, которые больше не нужны, и функции-члена `emplace()` для автоматического создания элемента в стеке (обе функции появились вместе со стандартом C++11).

```
stack<pair<string,string>> st;

auto p = make_pair("hello","world");
st.push(move(p)); // ОК, если p больше не используется

st.emplace("nico","josuttis");
```

12.1.3. Пользовательский класс стека

Стандартный класс `stack<>` пожертвовал удобством и безопасностью в пользу быстрой работы. Мне это не нравится, поэтому я написал свой собственный класс стека, имеющий два преимущества.

1. Функция-член `pop()` не только удаляет элемент с вершины стека, но и возвращает его.
2. Функции-члены `pop()` и `top()` генерируют исключение, если стек пуст.

Кроме того, я не стал включать в класс функции, которые не нужны рядовому пользователю, например операции сравнения. Мой класс стека определен следующим образом:


```

// contadapt/Stack.hpp
/* *****
 * Stack.hpp
 * - более безопасный и удобный класс стека
 * *****/
#ifndef STACK_HPP
#define STACK_HPP

#include <deque>
#include <exception>
template <typename T>

class Stack {
protected:
    std::deque<T> c; // контейнер для элементов

public:
    // класс исключения для функций-членов pop() и top() при пустом стеке
    class ReadEmptyStack : public std::exception {
    public:
        virtual const char* what() const throw() {
            return "read empty stack";
        }
    };

    // количество элементов
    typename std::deque<T>::size_type size() const {
        return c.size();
    }

    // пустой ли стек?
    bool empty() const {
        return c.empty();
    }

    // вносим элемент в стек
    void push (const T& elem) {
        c.push_back(elem);
    }

    // удаляем элемент из стека и возвращаем его значение
    T pop () {
        if (c.empty()) {
            throw ReadEmptyStack();
        }
        T elem(c.back());
        c.pop_back();
        return elem;
    }

    // возвращаем значение элемента на вершине стека
    T& top () {
        if (c.empty()) {

```

```
        throw ReadEmptyStack();
    }
    return c.back();
}
};
```

```
#endif /* STACK_HPP */
```

Для работы с этим классом предыдущий пример надо переписать следующим образом:

```
// contadapt/stack2.cpp
```

```
#include <iostream>
#include <exception>
#include "Stack.hpp"    // используем специальный стек класса class
using namespace std;
```

```
int main()
{
    try {
        Stack<int> st;

        // вносим три элемента в стек
        st.push(1);
        st.push(2);
        st.push(3);

        // удаляем и выводим на печать два элемента из стека
        cout << st.pop() << ' ';
        cout << st.pop() << ' ';

        // модифицируем элемент на вершине
        st.top() = 77;

        // добавляем новые элементы
        st.push(4);
        st.push(5);

        // убираем один элемент без обработки
        st.pop();

        // удаляем и выводим на печать три элемента
        // - ОШИБКА: одного элемента слишком много
        cout << st.pop() << ' ';
        cout << st.pop() << endl;
        cout << st.pop() << endl;
    }
    catch (const exception& e) {
        cerr << "EXCEPTION: " << e.what() << endl;
    }
}
```

Дополнительный заключительный вызов функции-члена `pop()` вызывает ошибку. В отличие от стандартного класса стека он генерирует исключение, а не приводит к неопределенному поведению. Программа выводит следующий результат:

```
3 2 4 77
EXCEPTION: read empty stack
```

12.1.4. Подробное описание класса `stack<>`

Интерфейс класса `stack<>` более или менее соответствует членам внутреннего контейнера. Рассмотрим пример:

```
namespace std {
    template <typename T, typename Container = deque<T>>
    class stack {
    public:
        typedef typename Container::value_type      value_type;
        typedef typename Container::reference        reference;
        typedef typename Container::const_reference  const_reference;
        typedef typename Container::size_type        size_type;
        typedef Container                            container_type;
    protected:
        Container c; // контейнер
    public:
        bool      empty() const           { return c.empty(); }
        size_type size() const           { return c.size(); }
        void      push(const value_type& x) { c.push_back(x); }
        void      push(value_type&& x)     { c.push_back(move(x)); }
        void      pop()                  { c.pop_back(); }
        value_type& top()                 { return c.back(); }
        const value_type& top()            const { return c.back(); }
        template <typename... Args>
        void      emplace(Args&&... args) {
            c.emplace_back(std::forward<Args>(args)...); }
        void      swap(stack& s) ... { swap(c,s.c); }
        ...
    };
}
```

Подробное описание функций-членов и операций приведено в разделе 12.4.

12.2. Очереди

Класс `queue<>` реализует очередь (также известную как FIFO). С помощью функции-члена `push()` в очередь можно вставить любое количество элементов (рис. 12.3). С помощью функции-члена `pop()` из очереди можно удалять элементы в том же порядке, в котором они были вставлены (“первым вошел, первым вышел”). Таким образом, очередь можно использовать в качестве классического буфера данных.

Для использования очереди в программу необходимо вставить заголовочный файл `<queue>`:

```
#include <queue>
```



Рис. 12.3. Интерфейс очереди

В заголовочном файле `<queue>` класс `queue` определен следующим образом:

```
namespace std {
    template <typename T,
              typename Container = deque<T>>
              class queue;
}
```

Первый шаблонный параметр — это тип элементов. Необязательный второй шаблонный параметр определяет контейнер, который очередь использует для хранения своих элементов. По умолчанию для этого используется дек.

Например, следующее объявление определяет очередь строк:

```
std::queue<std::string> buffer;    // очередь строк
```

Реализация очереди просто отображает операции в соответствующие вызовы контейнера, который используется в очереди (рис. 12.4). Можно использовать любой последовательный контейнерный класс, имеющий функции-члены `front()`, `back()`, `push_back()` и `pop_front()`. Например, в качестве контейнера можно использовать список:

```
std::queue<std::string, std::list<std::string>> buffer;
```

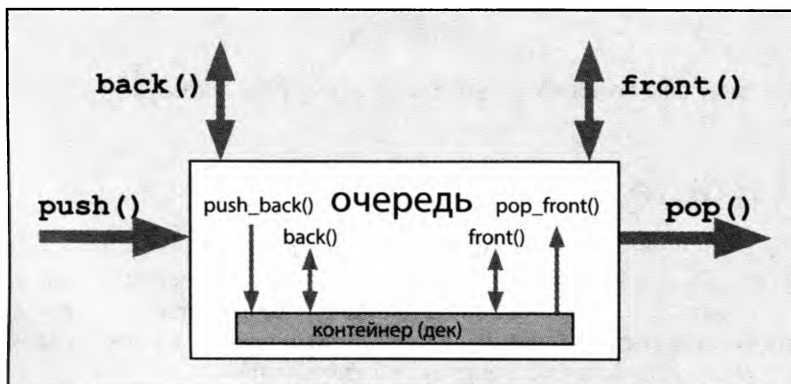


Рис. 12.4. Внутренний интерфейс очереди

12.2.1. Основной интерфейс

Основной интерфейс состоит из функций-членов `push()`, `front()`, `back()` и `pop()`.

- Функция-член **`push()`** вставляет элемент в очередь.
- Функция-член **`front()`** возвращает следующий доступный элемент в очереди (элемент, который был вставлен первым).
- Функция-член **`back()`** возвращает последний доступный элемент в очереди (элемент, который был вставлен последним).
- Функция-член **`pop()`** удаляет из очереди первый элемент.

Отметим, что функция-член `pop()` удаляет первый элемент, но не возвращает его, а функции-члены `front()` и `back()` возвращают элемент, не удаляя его. Таким образом, для обработки и удаления первого элемента очереди всегда следует вызывать функции-члены `front()` и `pop()`. Этот интерфейс несколько неудобный, но он работает лучше, если нужно только удалить следующий элемент, не обрабатывая его. Отметим, что поведение функций-членов `front()`, `back()` и `pop()` для пустой очереди не определено. Функции-члены `size()` и `empty()` проверяют, содержит ли очередь элементы.

Если вам не нравится стандартный интерфейс класса `queue<>`, можно легко написать более удобный интерфейс. Пример такого интерфейса приведен в разделе 12.2.3.

12.2.2. Пример использования очереди

Следующая программа демонстрирует использование класса `queue<>`:

```
// contadapt/queue1.cpp

#include <iostream>
#include <queue>
#include <string>
using namespace std;

int main()
{
    queue<string> q;

    // вставляем в очередь три элемента
    q.push("These ");
    q.push("are ");
    q.push("more than ");

    // читаем и выводим на печать два элемента из очереди
    cout << q.front();
    q.pop();
    cout << q.front();
    q.pop();

    // вставляем два новых элемента
    q.push("four ");
    q.push("words!");
```

```
// пропускаем один элемент
q.pop();

// читаем, выводим на печать и удаляем из очереди два элемента
cout << q.front();
q.pop();
cout << q.front() << endl;
q.pop();

// выводим на печать количество элементов в очереди
cout << "number of elements in the queue: " << q.size()
    << endl;
}
```

Программа выводит следующий результат:

```
These are four words!
number of elements in the queue: 0
```

12.2.3. Пользовательский класс очереди

Стандартный класс `queue<>` пожертвовал удобством и безопасностью в пользу быстрой работы. Это нравится не всем. Но можно легко написать свой собственный класс очереди, как показано выше при описании класса стека (раздел 12.1.3). Соответствующий пример представлен на веб-сайте, посвященном книге.

12.2.4. Подробное описание класса `queue` <>

Интерфейс класса `queue<>` более или менее соответствует членам внутреннего контейнера (см. раздел 12.1.4, в котором описаны соответствующие члены класса `stack<>`). Подробное описание членов и операций класса приведено в разделе 12.4.

12.3. Очереди с приоритетами

Класс `priority_queue<>` реализует очередь, из которой элементы считываются в соответствии с их приоритетом. Интерфейс такого класса похож на интерфейс очереди. Иначе говоря, функция-член `push()` вставляет элемент в очередь, а функции-члены `top()` и `pop()` обеспечивают доступ к следующему элементу и удаляют его (рис. 12.5). Однако следующий элемент — это не тот элемент, который был вставлен первым, а элемент, имеющий наивысший приоритет. Таким образом, элементы частично упорядочиваются по значениям. Как обычно, критерий сортировки можно задать в виде шаблонного параметра. По умолчанию элементы упорядочиваются с помощью оператора `<` в убывающем порядке.

Таким образом, следующий элемент — это всегда “наибольший” элемент. Если таких элементов несколько, то какой из них будет следующим, не определено.

Очереди с приоритетами определены в том же самом заголовочном файле, что и обычные очереди `<queue>`.

```
#include <queue>
```

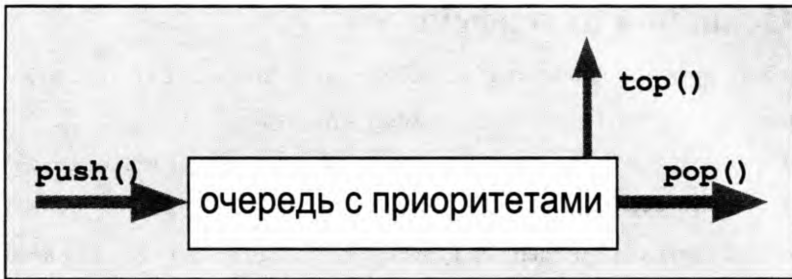


Рис. 12.5. Интерфейс очереди с приоритетами

В заголовочном файле `<queue>` класс `priority_queue` определен следующим образом:

```
namespace std {
    template <typename T,
              typename Container = vector<T>,
              typename Compare = less<typename Container::value_type>>
    class priority_queue;
}
```

Первый шаблонный параметр — это тип элементов. Необязательный второй шаблонный параметр определяет контейнер, который используется в очереди с приоритетами для хранения элементов. По умолчанию таким контейнером является вектор. Необязательный третий параметр определяет критерий сортировки, используемый для поиска следующего элемента с наивысшим приоритетом. По умолчанию этот параметр сравнивает элементы с помощью оператора `<`. Например, следующее определение объявляет очередь с приоритетами чисел типа `float`:

```
std::priority_queue<float> pbuffer; // очередь с приоритетами чисел типа float
```

Реализация очереди с приоритетами просто отображает операции в соответствующие вызовы контейнера, который используется в классе. Можно использовать любой последовательный контейнерный класс, имеющий итераторы произвольного доступа и функции-члены `front()`, `push_back()` и `pop_back()`. Произвольный доступ необходим для сортировки элементов, которая выполняется с помощью алгоритмов работы с пирамидами из библиотеки STL (см. раздел 11.9.4). Например, в качестве контейнера можно использовать дек:

```
std::priority_queue<float, std::deque<float>> pbuffer;
```

Для определения собственного критерия сортировки необходимо передать функцию, функциональный объект или лямбда-функцию в качестве бинарного предиката, используемого алгоритмами сортировки для сравнения двух элементов (более подробно критерии сортировки описаны в разделах 7.7.2 и 10.1.1). Например, следующее объявление определяет очередь с приоритетами с обратной сортировкой:

```
std::priority_queue<float, std::vector<float>,
                  std::greater<float>> pbuffer;
```

В этой очереди с приоритетами следующий элемент всегда является одним из наименьших.

12.3.1. Основной интерфейс

Основной интерфейс состоит из функций-членов `push()`, `top()` и `pop()`.

- Функция-член `push()` вставляет элемент в очередь.
- Функция-член `top()` возвращает следующий доступный элемент в очереди.
- Функция-член `pop()` удаляет элемент из очереди.

Как и во всех других контейнерных адаптерах, функция-член `pop()` удаляет следующий элемент, но не возвращает его, а функция-член `top()` возвращает элемент, не удаляя его. Таким образом, для обработки и удаления следующего элемента в очереди всегда следует вызывать обе эти функции-члены. Кроме того, как обычно, поведение функций-членов `top()` и `pop()` для пустой очереди не определено. Если есть сомнения, следует использовать функции-члены `size()` и `empty()`.

12.3.2. Пример использования очереди с приоритетами

Следующая программа демонстрирует использование класса `priority_queue<>`:

```
// contadapt/priorityqueue1.cpp

#include <iostream>
#include <queue>
using namespace std;

int main()
{
    priority_queue<float> q;

    // вставляем в очередь с приоритетами три элемента
    q.push(66.6);
    q.push(22.2);
    q.push(44.4);

    // читаем, выводим на печать и удаляем два элемента
    cout << q.top() << ' ';
    q.pop();
    cout << q.top() << endl;
    q.pop();

    // вставляем еще три элемента
    q.push(11.1);
    q.push(55.5);
    q.push(33.3);

    // пропускаем один элемент
    q.pop();

    // выводим на печать и удаляем остальные элементы
    while (!q.empty()) {
        cout << q.top() << ' ';
        q.pop();
    }
}
```



```

    }
    cout << endl;
}

```

Программа выводит следующий результат:

```

66.6 44.4
33.3 22.2 11.1

```

Как видим, после вставки чисел 66.6, 22.2 и 44.4 программа выводит на экран числа 66.6 и 44.4 как старшие элементы. После вставки еще трех элементов очередь с приоритетами содержит числа 22.2, 11.1, 55.5 и 33.3 (в порядке вставки). Следующий элемент пропускается с помощью вызова функции-члена `pop()`, поэтому заключительный цикл выводит на экран элементы 33.3, 22.2 и 11.1 в указанном порядке.

12.3.3. Подробное описание класса `priority_queue<>`

Очереди с приоритетами используют алгоритмы STL для работы с пирамидами.

```

namespace std {
    template <typename T, typename Container = vector<T>,
              typename Compare = less<typename Container::value_type>>
    class priority_queue {
    protected:
        Compare comp;    // критерий сортировки
        Container c;     // контейнер
    public:
        // конструкторы
        explicit priority_queue(const Compare& cmp = Compare(),
                               const Container& cont = Container())
            : comp(cmp), c(cont) {
            make_heap(c.begin(), c.end(), comp);
        }
        void push(const value_type& x) {
            c.push_back(x);
            push_heap(c.begin(), c.end(), comp);
        }
        void pop() {
            pop_heap(c.begin(), c.end(), comp);
            c.pop_back();
        }
        bool empty() const { return c.empty(); }
        size_type size() const { return c.size(); }
        const value_type& top() const { return c.front(); }
        ...
    };
}

```

Эти алгоритмы описаны в разделе 11.9.4.

Отметим, что в отличие от других контейнерных адаптеров, в очереди с приоритетами не определены операции сравнения. Подробное описание членов класса и операций приведено в разделе 12.4.

12.4. Подробное описание контейнерных адаптеров

Следующие подразделы содержат подробное описание членов и операций, определенных в контейнерных адаптерах `stack<>`, `queue<>` и `priority_queue<>`.

12.4.1. Определения типов

contadapt::value_type

- Тип элементов.
- Эквивалентно *container::value_type*.

contadapt::reference

- Тип ссылок на элементы.
- Эквивалентно *container::reference*.
- Доступно начиная со стандарта C++11.

contadapt::const_reference

- Тип ссылок на элементы, доступные только для чтения.
- Эквивалентно *container::const_reference*.
- Доступно начиная со стандарта C++11.

contadapt::size_type

- Целочисленный тип без знака для размерных значений.
- Эквивалентно *container::size_type*.

contadapt::container_type

- Тип контейнера.

12.4.2. Конструкторы

contadapt::contadapt ()

- Конструктор по умолчанию.
- Создает пустой стек или очередь (с приоритетами).

explicit contadapt::contadapt (const Container& cont)

explicit contadapt::contadapt (Container&& cont)

- Создает стек или очередь, инициализированную элементами контейнера *cont*, который должен быть объектом класса контейнерного адаптера.
- В первой форме все элементы контейнера *cont* копируются.
- Во второй форме все элементы контейнера *cont* перемещаются, если передаваемый контейнер поддерживает семантику перемещения, в противном случае элементы копируются (доступно начиная со стандарта C++11).
- Обе формы не поддерживаются классом `priority_queue<>`.

По стандарту C++11 все конструкторы допускают передачу в качестве дополнительного аргумента механизма распределения памяти, используемого для размещения внутреннего контейнера.

12.4.3. Вспомогательные конструкторы для очередей с приоритетами

`explicit priority_queue::priority_queue (const CompFunc& op)`

- Создает пустую очередь с приоритетами с предикатом *op*, используемым как критерий сортировки.
- Примеры, демонстрирующие передачу критерия сортировки в качестве аргумента конструктора, приведены в разделах 7.7.5 и 7.8.6.

`priority_queue::priority_queue (const CompFunc& op const Container& cont)`

- Создает очередь с приоритетами, которая инициализируется элементами контейнера *cont* и использует предикат *op* в качестве критерия сортировки.
- Все элементы контейнера *cont* копируются.

`priority_queue::priority_queue (InputIterator beg, InputIterator end)`

- Создает очередь с приоритетами, которая инициализируется всеми элементами диапазона *[beg, end)*.
- Эта функция является шаблонным членом (см. раздел 3.2), поэтому элементы диапазона-источника могут иметь любой тип, который может быть преобразован в тип элемента контейнера.

`priority_queue::priority_queue (InputIterator beg,
InputIterator end,
const CompFunc& op)`

- Создает очередь с приоритетами, которая инициализируется всеми элементами диапазона *[beg, end)* и использует предикат *op* как критерий сортировки.
- Эта функция является шаблонным членом (см. раздел 3.2), поэтому элементы диапазона-источника могут иметь любой тип, который может быть преобразован в тип элемента контейнера.
- Примеры, демонстрирующие передачу критерия сортировки в качестве аргумента конструктора, приведены в разделах 7.7.5 и 7.8.6.

`priority_queue::priority_queue (InputIterator beg, InputIterator end,
const CompFunc& op, const Container& cont)`

- Создает очередь с приоритетами, которая инициализируется всеми элементами контейнера *cont* и диапазона *[beg, end)* и использует предикат *op* как критерий сортировки.
- Эта функция является шаблонным членом (см. раздел 3.2), поэтому элементы диапазона-источника могут иметь любой тип, который может быть преобразован в тип элемента контейнера.

По стандарту C++11 все конструкторы допускают передачу в качестве дополнительного аргумента механизма распределения памяти, используемого для размещения внутреннего контейнера.

12.4.4. Операции

`bool contadapt::empty () const`

- Проверяет, пуст ли контейнерный адаптер (т.е. не содержит ни одного элемента).
- Эквивалентно `contadapt::size ()==0`, но может работать быстрее.

`size_type contadapt::size () const`

- Возвращает текущее количество элементов.
- Для проверки, является ли контейнерный адаптер пустым (т.е. не содержит ни одного элемента), следует использовать функцию-член `empty ()`, потому что она может работать быстрее.

`void contadapt::push (const value_type& elem)`

`void contadapt::push (value_type&& elem)`

- Первая форма вставляет копию аргумента *elem*.
- Первая форма перемещает аргумент *elem*, если поддерживается семантика перемещения, в противном случае аргумент *elem* копируется (начиная со стандарта C++11).

`void contadapt::emplace (args)`

- Вставляет новый элемент, инициализированный списком аргументов *args*.
- Доступно начиная со стандарта C++11.

`reference contadapt::top ()`

`const_reference contadapt::top () const`

`reference contadapt::front ()`

`const_reference contadapt::front () const`

- Все формы возвращают очередной элемент.
 - Для стека предусмотрены обе формы функции-члена `top ()`, возвращающие элемент, вставленный последним.
 - Для очереди предусмотрены обе формы функции-члена `front ()`, возвращающие элемент, вставленный первым.
 - Для очереди с приоритетами предусмотрена только вторая форма функции-члена `top ()`, возвращающая элемент с максимальным значением. Если максимальное значение имеют несколько элементов, стандарт не определяет, какой именно элемент должен быть возвращен.
- Вызывающая сторона должна гарантировать, что контейнерный адаптер содержит хотя бы один элемент (`size ()>0`), в противном случае последствия непредсказуемы.
- Форма, возвращающая неконстантную ссылку, позволяет модифицировать следующий доступный элемент, если он содержится в стеке или очереди. Насколько это соответствует хорошему стилю программирования, решать вам.
- До принятия стандарта C++11 типом возвращаемого значения был `(const) value_type&`, что обычно одно и то же.

```
void contadapt::pop ()
```

- Удаляет следующий элемент из контейнерного адаптера.
 - Для стека следующим является элемент, вставленный последним.
 - Для очереди следующим является элемент, вставленный первым.
 - Для очереди с приоритетами следующим является элемент, имеющий максимальное значение. Если максимальных элементов несколько, стандарт не определяет, какой из них следует удалить.
- Эта функция ничего не возвращает. Для обработки следующего элемента сначала необходимо вызвать функцию-член `top()` или `front()`.
- Вызывающая сторона должна гарантировать, что контейнерный адаптер содержит хотя бы один элемент (`size() > 0`), в противном случае последствия непредсказуемы.

```
reference queue::back ()
```

```
const reference queue::back () const
```

- Обе формы возвращают последний элемент очереди. Последним считается элемент, который был вставлен после всех остальных элементов очереди.
- Вызывающая сторона должна гарантировать, что контейнерный адаптер содержит хотя бы один элемент (`size() > 0`), в противном случае последствия непредсказуемы.
- Первая форма для неконстантных очередей возвращает ссылку. Следовательно, можно модифицировать последний элемент, пока он пребывает в очереди. Насколько это соответствует хорошему стилю программирования, решать вам.
- До принятия стандарта C++11 типом возвращаемого значения был `(const) value_type&`, что обычно одно и то же.
- Предусмотрена только для класса `queue<>`.

```
bool comparison (const contadapt& stack1, const contadapt& stack2)
```

- Возвращает результат сравнения двух стеков или очередей одного и того же типа.
- Оператор *comparison* может быть одной из следующих:


```
== и !=
<, >, <= и >=
```
- Два стека или очереди одинаковы, если они имеют одинаковое количество элементов, расположенных в одинаковом порядке (все сравнения двух соответствующих элементов должны возвращать значение `true`).
- Для проверки того, что один из стеков или одна из очередей меньше другого стека или очереди, контейнерные адаптеры сравниваются лексикографически (см. раздел 11.5.4).
- Не предусмотрена для класса `priority_queue<>`.

```
void contadapt::swap (contadapt& c)
```

```
void swap (contadapt& c1, contadapt& c2)
```

- Обменивает содержимое объекта `*this` с содержимым объекта `c`, а также содержимое объектов `c1` и `c2` соответственно. Для очередей с приоритетами меняет местами также и критерии сортировки.
- Вызывает функцию-член `swap()` соответствующего контейнера (см. раздел 8.4).
- Доступна начиная со стандарта C++11.

12.5. Битовые множества

Битовые множества моделируют массивы фиксированного размера, состоящие из битов или булевых значений. Они полезны для управления наборами флагов, когда переменные могут представлять любую комбинацию флагов. Программы на языке С и старые программы на языке С++ для представления массивов битов обычно использовали тип `long` и манипулировали им с помощью побитовых операций, таких как `&`, `|` и `~`. Класс `bitset` имеет преимущество над типом `long`. Оно заключается в том, что битовые множества могут содержать произвольное количество битов и предусматривают дополнительные операции. Например, можно присваивать отдельные биты, а также читать и записывать битовые множества как последовательности, состоящие из 0 и 1.

Отметим, что количество битов в битовом множестве изменить нельзя. Количество битов задается шаблонным параметром. Если нужен контейнер с переменным количеством битов или булевых значений, следует использовать класс `vector<bool>` (описанный в разделе 7.3.6).

Класс `bitset` определен в заголовочном файле `<bitset>`.

```
#include <bitset>
```

В заголовочном файле `<bitset>` класс `bitset` определен как шаблонный класс, в котором количество битов задается шаблонным параметром.

```
namespace std {
    template <size_t Bits>
    class bitset;
}
```

В этом случае шаблонный параметр представляет собой не тип, а целое число без знака (см. раздел 3.2).

Шаблоны с разными шаблонными аргументами считаются разными типами. Сравнивать и комбинировать можно лишь битовые множества, содержащие одинаковое количество битов.

Новшества стандарта C++11

В стандарте C++98 были определены практически все свойства битовых множеств. Перечислим самые важные дополнения, появившиеся в стандарте C++11.

- Битовые множества теперь можно инициализировать строковыми литералами (см. раздел 12.5.1).
- Преобразования в числовые значения (и наоборот) теперь поддерживают работу с типом `unsigned long long`. Для этого была предложена функция `to_ullong()` (см. раздел 12.5.1).
- Преобразования в строки (и наоборот) теперь позволяют задавать символ, интерпретируемый как установленный и сброшенный бит.
- Появилась функция-член `all()`, проверяющая, все ли биты установлены.
- Для использования битовых множеств в неупорядоченных контейнерах предусмотрена хеш-функция по умолчанию (см. раздел 7.9.2).

12.5.1. Примеры использования битовых множеств

Использование битовых множеств в качестве наборов флагов

Первый пример демонстрирует, как использовать битовые множества для управления наборами флагов. Каждый флаг имеет значение, определенное перечислением. Значение перечислимого типа используется как позиция в битовом множестве. В частности, биты представляют цвета. Таким образом, каждое значение перечисления определяет один цвет. Используя битовые множества, можно управлять переменными, которые могут содержать любую комбинацию цветов.

```
// contadapt/bitset1.cpp

#include <bitset>
#include <iostream>
using namespace std;

int main()
{
    // тип перечисления для битов
    // - каждый бит представляет цвет
    enum Color { red, yellow, green, blue, white, black, ...,
                numColors };

    // создаем битовое множество для всех битов/цветов
    bitset<numColors> usedColors;

    // устанавливаем биты для двух цветов
    usedColors.set(red);
    usedColors.set(blue);

    // выводим на печать данные битового множества
    cout << "bitfield of used colors: " << usedColors << endl;
    cout << "number of used colors: " << usedColors.count() << endl;
    cout << "bitfield of unused colors: " << ~usedColors << endl;

    // если использован хотя бы один цвет
    if (usedColors.any()) {
        // цикл по всем цветам
        for (int c = 0; c < numColors; ++c) {
            // если использован текущий цвет
            if (usedColors[(Color)c]) {
                ...
            }
        }
    }
}
```

Использование битовых множеств для ввода и вывода битовых представлений

Полезным свойством битовых множеств является возможность преобразовывать целочисленные значения в последовательность битов, и наоборот. Это делается с помощью создания временного битового множества.

```
// contadap/bitset2.cpp

#include <bitset>
#include <iostream>
#include <string>
#include <limits>
using namespace std;

int main()
{
    // выводим несколько чисел в битовом представлении
    cout << "267 as binary short: "
         << bitset<numeric_limits<unsigned short>::digits>(267)
         << endl;
    cout << "267 as binary long: "
         << bitset<numeric_limits<unsigned long>::digits>(267)
         << endl;
    cout << "10000000 with 24 bits: "
         << bitset<24>(1e7) << endl;

    // записываем битовое представление в строку
    string s = bitset<42>(12345678).to_string();
    cout << "12345678 with 42 bits: " << s << endl;

    // преобразовываем бинарное представление в целое число
    cout << "\"1000101011\" as number: "
         << bitset<100>("1000101011").to_ullong() << endl;
}
```

В зависимости от количества битов в типах short и long long программа может выдать следующий результат:

```
267 as binary short: 0000000100001011
267 as binary long: 00000000000000000000000100001011
10000000 with 24 bits: 100110001001011010000000
12345678 with 42 bits: 00000000000000000000101111000110000101001110
"1000101011" as number: 555
```

В этом примере следующее выражение преобразовывает число 267 в битовое множество с количеством битов, заданным числом типа `unsigned short` (см. раздел 5.3, посвященный числовым пределам):

```
bitset<numeric_limits<unsigned short>::digits>(267)
```

Оператор вывода для класса `bitset` выводит биты как последовательность символов 0 и 1.

Битовые множества можно вывести непосредственно или использовать их значение как строку.

```
string s = bitset<42>(12345678).to_string();
```

До принятия стандарта C++11 здесь приходилось писать

```
string s = bitset<42>(12345678).to_string<char,char_traits<char>, allocator<char>>();
```


потому что `to_string()` — это шаблонная функция-член, а для шаблонных аргументов функций значения по умолчанию предусмотрены не были.

Аналогично следующее выражение преобразовывает последовательность бинарных символов в битовое множество, для которого функция-член `to_ullong()` выдает целое число:

```
bitset<100>("1000101011")
```

Отметим, что количество битов в битовом множестве не должно превышать `sizeof(unsigned long long)*8`. Причина заключается в том, что если битовое множество невозможно представить в виде числа типа `unsigned long long`, возникает исключение².

До принятия стандарта C++11 начальное значение необходимо было преобразовывать в тип `string` явным образом:

```
bitset<100>(string("1000101011"))
```

12.5.2. Подробное описание класса `bitset`

Из-за ограниченного объема книги подробное описание членов класса `bitset<>` размещено на веб-сайте, посвященном книге, по адресу <http://www.cppstdlib.com>.

² До принятия стандарта C++11 тип `unsigned long long` не был предусмотрен, поэтому здесь можно было вызывать только функцию `to_ulong()`. Эту функцию все еще можно вызывать, если количество битов меньше `sizeof(unsigned long)`.

